



Lab #1: Simple MOV and ADD

Objective

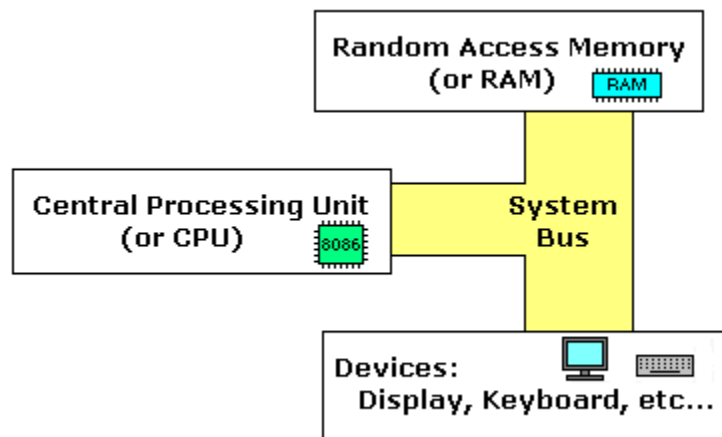
- To get introduced to the x86 registers
- To understand basic addressing modes using the emulator.
- To get introduced to the MOV instruction.

Introduction

Visual Studio's integrated development environment (IDE) provides tools for editing, assembling, and debugging assembly code, allowing you to monitor registers, flags, memory, and the stack in real-time. The Irvine library simplifies common tasks like string manipulation and I/O operations, making it easier for beginners to grasp core concepts. With step-by-step debugging, including breakpoints and ALU operation tracking, Visual Studio helps you understand the impact of each instruction on the system.

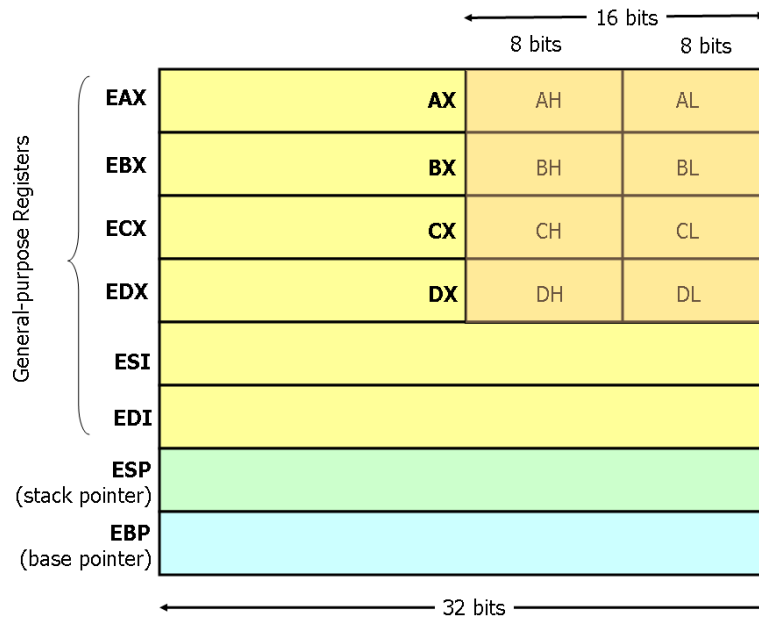
What is assembly language?

Assembly language is a low-level programming language. A simple computer model is shown below. The system bus (shown in yellow) connects the various components of a computer. The CPU is the heart of the computer, most of computations occur inside the CPU. RAM is a place where the programs are loaded in order to be executed.



The x86 CPU has several general-purpose registers, each with a specific name. In this lab, we will focus on the following registers:

- **EAX** - the accumulator register (can be accessed as AX, AH, and AL for lower 16-bit and 8-bit operations).
- **EBX** - the base register (can be accessed as BX, BH, and BL for lower 16-bit and 8-bit operations).
- **ECX** - the count register (can be accessed as CX, CH, and CL for lower 16-bit and 8-bit operations).
- **EDX** - the data register (can be accessed as DX, DH, and DL for lower 16-bit and 8-bit operations).



Basic Addressing Modes using Move Instruction

Instruction format:

MOV destination, source

Addressing Mode	Destination	Source	Example
Immediate	- General register (EAX, EBX, ECX, EDX) - Index/pointer register (ESI, EDI, EBP, ESP) - Memory address	Immediate value	MOV EAX, 1002h MOV AX, 1002h MOV AL, 02h
Register-Register	- General register (EAX, EBX, ECX, EDX) - Index/pointer register (ESI, EDI, EBP, ESP) - Memory address - Segment register (DS, SS, ES)	Register	MOV EAX, EBX MOV AX, BX MOV AL, BL

Things to keep in mind:

- When moving data between registers, both the source and destination must be of the same size (32-bit to 32-bit, 16-bit to 16-bit, or 8-bit to 8-bit)
- You can move an immediate value (constant) into a larger register, even if the immediate value is smaller. For example, `MOV EAX, 02h` (moves 8-bit value into 32-bit register). The upper bits of the register will be cleared.

Addition using ADD Instruction

Arithmetic	Op1	Op2	Example
add Op1, Op2	- General register (EAX, EBX, ECX, EDX) - Memory address	- General register (EAX, EBX, ECX, EDX) - Memory address - Immediate value	ADD EAX, EBX ADD AX, BX ADD AL, 0F0H

Useful Irvine Commands to print the register values

Command	Description	Example
DumpRegs	Displays the EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP, EIP, and EFL registers in hexadecimal. Also displays the Carry, Sign, Zero, and Overflow flags. EIP displays the offset of the instruction which follows the call to DumpRegs.	call DumpRegs
WriteBin	Writes a 32-bit integer to standard output in ASCII binary format. The binary bits are displayed in groups of 4 for easy reading. EAX = the integer to write	mov eax,1234ABCDh call WriteBin
WriteChar	Writes a single character to standard output. AL = the character to write	mov al','\$' call WriteChar
WriteDec	Writes an unsigned 32-bit decimal number to standard output in decimal format with no leading zeros. EAX = unsigned number to write	mov eax,123456 call WriteDec
WriteHex	Writes an unsigned 32-bit hexadecimal number to standard output in 8-digit hexadecimal format. Leading zeros are inserted if necessary. This procedure is useful for printing the result of a multi-double-word precision arithmetic operation. EAX = unsigned number to write	mov eax,7FFFFFFh call WriteHex
WriteInt	Writes a signed 32-bit decimal number to standard output in decimal format with a leading sign and no leading zeros. EAX = signed number to write	mov eax,216543 call WriteInt
WriteString	Writes a null-terminated string to standard output. EDX = points to string	.data prompt BYTE "Enter your name: ",0 .code

		<code>mov edx,OFFSET prompt</code> <code>call WriteString</code>
--	--	---

In Lab Work (Questions 1-4):

Use the following template for your code. Make sure to comment your code.

- Trace the following code and fill in the contents of each register after the execution of every line.
(No screenshots for this question)

Instruction	EAX	EBX	ECX	EDX
MOV AX, 1234H	00001234	00000000	00000000	004010AA
MOV BX, 0101B	00001234	00000005	00000000	004010AA
MOV CX, 1000D	00001234	00000005	000003E8	004010AA
MOV EAX, 0000FFFFH	0000FFFF	00000000	000003E8	004010AA
MOV EBX, 0BEEFCAFEH	0000FFFF	BEEFCAFE	00000000	004010AA
MOV DL, 0AH	0000FFFF	BEEFCAFE	00000000	0040100A
ADD AX, BX	0000CAFD	BEEFCAFE	00000000	0040100A
ADD EAX, 00001234H	0000DD31	BEEFCAFE	00000000	0040100A
ADD DL, CL	0000DD31	BEEFCAFE	00000000	0040100A
MOV DX, AX	0000DD31	BEEFCAFE	00000000	0040DD31
ADD EAX, 10000000H	1000DD31	BEEFCAFE	00000000	0040DD31

- Identify whether the following instructions are correct or wrong. If it is wrong write the correct instruction(s) to achieve the required functionality. Note that for some cases, the correct answer may consist of multiple instructions.

Instruction	Correct/Wrong	Correct instruction(s)
MOV AL, 04H	<u>Correct</u>	
MOV BX, 1234h MOV CL, BX	<u>Wrong</u>	MOV BX, 1234H MOV CX, BX Can MOV registers of different bit sized.
MOV BX, 0B5H	<u>Correct</u>	
ADD BX, 26H	<u>Correct</u>	
MOV CX, 1234h MOV BX, 5678h ADD BX, CL	<u>Wrong</u>	MOV CX, 1234h MOV BX, 5678h ADD BX, CX Can ADD registers of different bit sized.

- Write instructions to load 0ABCD123H and 0FEDCBA9H into the EAX and ECX registers, respectively, using Immediate-addressing mode. Then write an instruction that will load the value of EAX into ECX using register-register addressing mode. What is the new value of ECX?

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

; Your data goes here

```

.code
main PROC
    ; Your code goes here
    MOV EAX, 0ABCD123H
    MOV ECX, 0FEDCBA9H
    MOV ECX, EAX

    call DumpRegs

    exit
main ENDP
END main

```

```

EAX=0ABCD123  EBX=0024A000  ECX=0ABCD123  EDX=004010AA
ESI=004010AA  EDI=004010AA  EBP=0019FF80  ESP=0019FF74
EIP=00403671  EFL=00000246  CF=0  SF=0  ZF=1  OF=0  AF=0  PF=1

```

4. Load EAX and EBX with 11091109 and 24042404 respectively. Write a set of instruction to swap the two registers. Then display the sum of the two registers in EDX.
5. TITLE Name/Purpose Here
- 6.
7. INCLUDE Irvine32.inc
- 8.
9. .data
10. ; Your data goes here
- 11.
- 12.
13. .code
14. main PROC
15. ; Your code goes here
16. MOV EAX, 11091109
17. MOV EBX, 24042404
18. MOV EDX, EBX
19. MOV EBX, EAX
20. MOV EAX, EDX
21. ADD EDX, EAX
- 22.
23. call DumpRegs
- 24.
25. exit
26. main ENDP
27. END main

```

EAX = 016EDBA4 EBX = 00A93CA5 ECX = 004010AA EDX = 02DDB748 ESI = 004010AA
EDI = 004010AA EIP = 00403672 ESP = 0019FF74 EBP = 0019FF80 EFL = 00000206

```

```

EAX=016EDBA4  EBX=00A93CA5  ECX=004010AA  EDX=02DDB748
ESI=004010AA  EDI=004010AA  EBP=0019FF80  ESP=0019FF74
EIP=00403677  EFL=00000206  CF=0  SF=0  ZF=0  OF=0  AF=0  PF=1

```